Source: docs\ADMIN_DASHBOARD.md | Export: 11 November 2026

Administrator dashboard

Local implementation only. Existing uncommitted customer purchase/checkout work is preserved. No production records, rules, indexes or deployments were changed.

Design and routes

The desktop Stitch reference supplies the hierarchy: four summary cards, Recent Orders and Low Stock. Mobile uses the same four cards with order/stock cards instead of narrow tables. The explicit brief's Lora typography and maroon/ivory/gold palette take precedence over the mixed font families and simulated phone wrapper in the reference files. No sample identities, amounts, counters, activity indicators or stock records ship in application code.

Admin styles are scoped to `.parinaya-admin` and dashboard-specific classes. Lora is self-hosted in `public/fonts/Lora-Variable.ttf`; its SIL Open Font License is included beside it. Icons in the active admin shell, product list and notifications use SVGs. Customer storefront styles, media and authentication are unchanged by this task.

The shared desktop sidebar is 240px. Below 1024px it becomes a modal navigation drawer with backdrop dismissal, Escape, Tab/Shift+Tab containment, initial/return focus and body scroll locking. It closes after navigation. Browser zoom remains enabled. The sidebar uses the required thirteen destinations in order, with View Website and Sign Out as bottom utilities.

Existing `/admin`, `/admin/dashboard`, `/admin/login` dashboard destinations and all navigation aliases remain. Marketing uses the existing `/admin/marketing` route and `/admin/coupons` alias. Product/order detail routes remain unchanged. About Us and Customer Enquiries retain their honest unimplemented states.

Metrics and exact meanings

Card	Authorized server data	Scope
New Orders	`dashboard:period`: `orders` count, `status == Placed`, `createdAt >= from`, `createdAt <= to`	Selected creation-date period
To Pack	`dashboard:backlog`: existing report aggregate's `Confirmed` count	Current backlog, all dates
Processing	`dashboard:backlog`: existing report aggregate's `Processing` count	Current backlog, all dates
COD to Collect	Active due-order aggregate `reportingPaise - collectedPaise`	Current outstanding amount, all dates

`Confirmed` is eligible to enter processing. `Processing` is **not** proof that packing is complete or dispatch is ready. The current engine has no Packed or Ready to Dispatch state, so the dashboard accurately labels the third card Processing. Existing status transitions are preserved; no new business state is introduced.

Outstanding COD includes `paymentStatus == due` only for Placed, Confirmed, Processing, Shipped and Delivered. Cancelled/Rejected and collected orders are excluded. Existing partial collections reduce the amount owed. Refunds describe returning previously collected funds and do not create new delivery debt. Delivered remains due until the existing independent collect action records collection. No order value is labelled collected revenue.

The dashboard reuses the existing report's sum/count completeness checks and its canonical aggregate field ordering. The historical `reports:get.duePaise` field retains its previous gross-due semantics; a separate `outstandingPaise` field provides the dashboard's amount after recorded collections. Unsafe or negative outstanding sums fail explicitly instead of publishing misleading values.

Today is the current India calendar day; Last 7 Days is today and the preceding six calendar days; This Month is the full current calendar month. Custom dates are inclusive. Bounds are calculated at Asia/Kolkata midnight (UTC+05:30), independently of the browser/server's local zone. The selected period filters New Orders and Recent Orders only. Card captions and dashboard explanatory text explicitly identify the other cards as all-date current backlog.

Recent orders and bulk actions

`dashboard:orders` uses the existing order-list builder with a server-enforced five-row limit, descending creation time/document-ID tie-break, date range and optional existing status filter. Its response includes only ID/reference, current status/revision, creation time, total, snapshot customer name and phone. Street addresses, account email/UID, notes and other order internals are not sent to the dashboard. Missing historical phone/name fields have

explicit fallbacks.

New maps to Placed, To Pack maps to Confirmed, and Processing retains its real status. Retry retains the selected filter. Request cancellation and generation/key checks prevent older responses from replacing a newer filter or another account's data.

Selection is limited to the currently displayed eligible rows. Bulk controls appear only after selection. Confirm orders is available for Placed rows; Start processing for Confirmed rows. Mixed states without a shared supported transition require a narrower selection. Shipping, cancellation, refund and collection are not offered as simplistic dashboard batch buttons; their existing detail workflows retain required inputs and safeguards.

The confirmation dialog freezes each selected order's revision and one operation ID. Each call uses existing `adminOrders:update`, including both administrator checks and its transaction/audit/operation-receipt handling. Requests run serially behind a synchronous submission lock. Successful selections are removed; failed selections remain. Revision conflicts require review against refreshed data and a newly confirmed eligible action. Uncertain/network failures expose Retry uncertain actions safely, replaying the exact original operation IDs/bodies even if the first response was lost after acceptance. No order is mutated merely by selecting, refreshing or mounting a route.

Low stock

The existing `inventory:low` response now also includes revision and the saved cover-image URL. Each simple product contributes its actual simple quantity; each colour contributes only its own quantity. Rows are included only at or below that product's actual threshold. The dashboard shows at most five low/out-of-stock rows from the existing bounded 200-product scan. It explicitly reports scan coverage and whether more products remain, and never presents a page count as a global inventory total.

Update Stock opens the existing `/admin/products/edit/{id}` editor. Its stock-adjustment form continues to enforce revision checks, reasons and inventory audit records. View Inventory opens the existing inventory route. Add Product, order View and View All Orders likewise use existing functional routes.

Refresh, errors and authorization

The shared admin data provider polls only visible tabs every 30 seconds. Visibility/focus can trigger a refresh after five seconds of staleness. Successful existing administrator mutations send a data-free same-window event and same-account BroadcastChannel signal; refreshes are debounced by 200ms. No order/customer payload, token or credentials are broadcast. This gives immediate cross-tab feedback for admin changes, with polling covering changes from customers or external processes. The interface reports actual last-successful refresh times in IST rather than claiming a live connection.

Metrics, recent orders, stock and notifications have independent API requests/errors/retries. Each resource allows one active request, cancels stale filters/unmounts and checks account/request identity before accepting results. Background refresh retains the last successful result and labels refresh failures. Initial loads show skeletons; failure shows an unavailable dash, never zero. Zero appears only after a successful response confirms it. Provider intervals/listeners/channels and pending reads are cleaned up on unmount/sign-out/account switch.

The notification panel starts closed, uses the existing real unread count/read/navigation API, and does not open during refresh or tab focus. Route navigation closes it. Escape/outside click close it; mobile positioning stays inside the viewport.

All new `dashboard:*` actions pass the existing server administrator gate: a verified token containing `admin: true` **and** an enabled `adminAccess/{uid}` record. Mutations recheck enabled access inside the existing transaction. Resource authorization errors clear protected results and invoke the existing permission watcher. Ordinary background refresh does not recreate the Firebase session or show permission-check overlays.

Index and rule dependencies

No Firestore manifest/rule changes are required or prepared. The new query shapes use existing definitions:

Query	Collection/scope	Ordered fields
Period Placed count	orders / COLLECTION	status ASC, createdAt ASC
Filtered recent orders with dates	orders / COLLECTION	status ASC, createdAt DESC, implicit document ID DESC
Unfiltered recent orders	orders / COLLECTION	createdAt DESC, implicit document ID DESC (automatic single-field index)
Current report aggregates/counts	orders / COLLECTION	Unchanged shapes documented in REPORT_INDEX_AUDIT.md

The two undated canonical sum indexes remain `status, collectedPaise, refundPaise, reportingPaise, __name__` and `paymentStatus, status, collectedPaise, refundPaise, reportingPaise, __name__`, all ascending. All previously defined dated aggregates, count indexes and other collection indexes remain untouched. The query-builder regression verifies count/recent coverage and that backlog issues exactly the existing report queries.

This task does not recheck production index metadata. Prior publication/readiness evidence belongs to the earlier index audit/provisioning task; emulator and local manifest success cannot establish current hosted index availability. No production credential or data migration is needed for the local dashboard implementation.

Changed files

- Shared shell: `src/admin/AdminLayout.tsx`, `src/admin/admin.css`, `src/admin/AdminIcon.tsx`, `src/admin/dialog.ts`, `src/admin/AdminPanel.tsx`, `src/admin/AdminLogin.tsx`, `src/admin/AdminAccess.tsx`, `src/admin/AdminProducts.tsx`.
- Dashboard/data: `src/admin/AdminDashboard.tsx`, `src/admin/dashboard/{model.ts,Dates.tsx,RecentOrders.tsx,Status.tsx,Notifications.tsx,resource.tsx}`, `src/admin/changeSignal.ts`.
- Existing API/client extensions: `server/business.ts`, `server/orders.ts`, `server/catalog.ts`, `src/products/client.ts`, `src/orders/AdminBusiness.tsx`.
- Tests/assets/docs: `src/admin/dashboard/model.test.ts`, `scripts/dashboard-emulator.test.ts`, `scripts/dashboard-browser.test.py`, `scripts/dashboard-fixture-control.ts`, `scripts/catalog-indexes.test.ts`, `scripts/admin-browser.test.py`, `scripts/catalog-recovery-browser.test.py`, `scripts/admin-emulator-suite.ts`, `package.json`, `public/fonts/Lora-Variable.ttf`, `public/fonts/Lora-OFL.txt`, this document.

Local acceptance

Run `npm.cmd run lint`, `npm.cmd test`, `npm.cmd run build`, and `npm.cmd run test:admin`. The last command starts isolated loopback Auth/Firestore emulators, creates temporary accounts/fixtures, runs API and browser suites, and shuts the emulators down. `scripts/dashboard-browser.test.py` performs the dashboard acceptance steps automatically with real local authorization and mutation APIs; it does not need production credentials. The optional pinned `npm.cmd run test:catalog-runtime` harness checks a locally packaged function.

The dashboard test verifies current/period metrics, empty and partial-error states, date controls, stale responses, mixed/conflicted bulk work, lost-response receipt replay, two real admin tabs, audited inventory adjustment, Delivered versus recorded collection, notifications, sign-out/customer rejection and normal-zoom 320/390/768/1024/1440/1920px captures. Screenshots and logs are ignored files under `.cache`. The final validation results are recorded below after the checks complete.

Validation completed

- Typecheck, all 78 unit tests, production frontend build and `git diff --check`: passed on the final source.
- All 11 API/rules/emulator suites passed, including the new exported-handler dashboard regression and the existing admin/category/profile, catalog, appearance, shopping, delivery, shipping, COD/order and business checks.
- All 13 browser suites passed across the full-run attempt and targeted reruns: customer authentication, login appearance, administrator access, categories, products, shopping, delivery, shipping, business, catalog recovery, purchase journey, COD and dashboard. The full run was interrupted when a local source edit caused Vite hot reload to unmount a category dialog. Category through COD were rerun successfully without further application edits during those tests. The dashboard fixture assertion was then corrected to allow other valid catalog fixtures in the bounded low-stock list; the final focused dashboard run passed. A final uninterrupted `test:admin` invocation was not repeated after the targeted passes.
- The final dashboard browser run also passed explicit Custom-range reopening, a change arriving while an aggregate request is held, hidden-tab deferral/return, stale filter cancellation, partial bulk conflicts, exact operation-ID replay after a lost response, and listener cleanup followed by an ordinary customer's rejected login.
- Screenshots were captured at 320, 390, 768, 1024, 1440 and 1920px with `visualViewport.scale == 1` and no horizontal page overflow. Mobile, tablet/sidebar and wide-desktop captures were visually inspected. Drawer keyboard focus, Escape, backdrop implementation and scroll restoration were checked; no automatic notification overlay appeared during refresh/focus.
- Packaged function checks passed using pinned Vercel builder 24.0.3, a `nodejs24.x` artifact and local Node 22.22.2, without a `tsx` runtime loader, with require(ESM) disabled and enabled. This is local packaging/runtime evidence, not verification of execution in hosted Node 24.
- No new rules or index definitions were prepared. Existing manifests are unchanged. Current production index metadata and hosted dashboard operation were not checked by this task.
- No commit, push, Vercel deployment, Firebase publication, production-record mutation or deletion of the owner's Test product occurred. All mutations used isolated emulator fixtures.